



MÓDULO 2

5 HORAS

Elementos Básicos de la Programación Estructurada con Python

Domina los fundamentos que todo programador necesita: tipos de datos, variables, operadores y expresiones. Un recorrido práctico y dinámico para construir tu base sólida en Python.

Curso para DTIC · UNAM · Autor: MC Manuel Maldonado

Objetivos del Módulo

Al completar estas 5 horas de formación, serás capaz de escribir expresiones complejas en Python, comprender cómo el intérprete evalúa cada operación y representar soluciones mediante pseudocódigo y diagramas de flujo.

01

Tipos de datos primitivos

Comprender `int`, `float`, `str` y `bool`, y saber cuándo usar cada uno.

02

Variables, constantes y palabras reservadas

Manejar la memoria del programa con buenas prácticas de nomenclatura.

03

Operadores y expresiones

Construir expresiones con operadores aritméticos, relacionales y lógicos.

04

Jerarquía y evaluación

Aplicar la precedencia de operadores para predecir resultados correctamente.

05

Pseudocódigo y algoritmos

Representar soluciones estructuradas antes de escribir código.

Tipos de Datos Primitivos

Los tipos de datos son la materia prima de cualquier programa. En Python, cada valor tiene un tipo que determina qué operaciones puedes realizar con él. Python es un lenguaje de **tipado dinámico**: no necesitas declarar el tipo de una variable, el intérprete lo infiere automáticamente al asignarle un valor.



Numéricos

int y float para cantidades enteras y decimales



Cadenas (str)

Texto entre comillas simples o dobles



Booleanos

True y False para valores de verdad

Numéricos: Enteros y Flotantes

Enteros (int)

Representan números sin decimales. Python maneja enteros de precisión arbitraria, lo que significa que pueden ser tan grandes como la memoria lo permita. Son ideales para contadores, índices y cantidades exactas.

```
edad = 25
cantidad = -10
print(type(edad)) # <class 'int'>
```

Flotantes (float)

Representan números con decimales (punto flotante). Utilizan el estándar IEEE 754 de doble precisión. Son ideales para mediciones, precios y cálculos científicos, aunque pueden presentar pequeños errores de redondeo.

```
precio = 99.99
temperatura = -5.5
print(type(precio)) # <class 'float'>
```

 **Actividad interactiva:** ¿Qué tipo de dato usarías para representar la edad de una persona? ¿Y para el precio de un producto? Prueba en tu consola de Python y observa cómo el intérprete infiere el tipo automáticamente con `type()`.

Cadenas de Texto (str)

Las cadenas representan texto y son uno de los tipos más utilizados en programación. En Python se definen entre comillas simples `'` o dobles `"`. A diferencia de otros lenguajes, no existe un tipo `char` separado: un carácter individual es simplemente una cadena de longitud 1.

Definición básica

```
nombre = "Manuel"
letra = 'A'
mensaje = 'Hola, mundo!'
print(type(nombre))
# <class 'str'>
```

Caracteres especiales

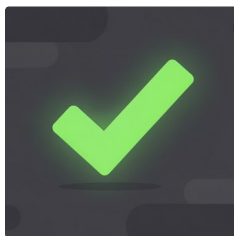
Python incluye secuencias de escape para caracteres que no pueden escribirse directamente:

- `\n` → nueva línea
- `\t` → tabulación
- `\\` → barra invertida

```
print("Línea 1\nLínea 2")
```

Booleanos: La Base de la Lógica

Los valores booleanos son el fundamento de todas las decisiones en programación. Solo existen dos valores posibles: True (verdadero) y False (falso). Son esenciales para las estructuras de control como if, while y for.



True (Verdadero)

Representa una condición que se cumple. Se genera al comparar valores que satisfacen la condición, por ejemplo: `5 > 3` produce True.



False (Falso)

Representa una condición que no se cumple. Valores como 0, cadenas vacías "" y None también se evalúan como False en contextos booleanos.

```
es_mayor = True
tiene_permiso = False
print(type(es_mayor)) # <class 'bool'>
print(5 > 3) # True
```

Palabras Reservadas en Python

Las **palabras reservadas** (keywords) son identificadores que tienen un significado especial para el intérprete de Python. No pueden utilizarse como nombres de variables, funciones ni clases. Python cuenta con aproximadamente **35 palabras reservadas** que forman la gramática del lenguaje.

and as assert async await
break class continue

def del elif else except False
finally for

from global if import in is
lambda None

nonlocal not or pass raise
return True try while with
yield

Incorrecto — provoca un error de sintaxis:

```
# if = 10
```

Correcto:

```
condicion = 10
```

Consulta la lista completa:

```
import keyword
```

```
print(keyword.kwlist)
```

Variables: Espacios de Memoria con Nombre

Las variables son **espacios de memoria** que almacenan datos y se identifican con un nombre. En Python, las variables se crean al asignarles un valor con el operador `=`. Gracias al **tipado dinámico**, no es necesario declarar el tipo previamente.

1

Letras, números y `_`

Los nombres pueden contener estos caracteres, pero **nunca deben comenzar con un número**.

2

Sensibles a mayúsculas

`Edad` y `edad` son variables completamente distintas.

3

Sin palabras reservadas

No uses `if`, `for`, `class`, etc. como nombre de variable.

4

snake_case

Convención recomendada: `total_ventas` en lugar de `tv` o `TotalVentas`.

```
nombre = "Ana"
edad = 30
precio = 19.99
edad = 31 # Reasignación válida
```


Constantes: Valores que No Cambian

Las constantes representan valores fijos durante toda la ejecución del programa. A diferencia de otros lenguajes como Java o C, **Python no tiene un mecanismo nativo para definir constantes**. Sin embargo, la comunidad utiliza una convención universal: escribir el nombre **completamente en mayúsculas** para indicar que ese valor no debe modificarse.

Ejemplo de constantes

```
PI = 3.1416
GRAVEDAD = 9.8
VELOCIDAD_LUZ = 299792458
```

```
# Por convención, no se modifican
radio = 5
area = PI * radio ** 2
print(area) # 78.54
```

¿Cuándo usar constantes?

- Valores matemáticos (PI, e, phi)
- Configuraciones fijas del programa
- Límites y umbrales predefinidos
- Factores de conversión de unidades

Actividad: Define las constantes necesarias para calcular el área de un círculo y la fuerza gravitatoria.

Expresiones y Operadores

Una **expresión** es una combinación de valores, variables, operadores y llamadas a funciones que Python evalúa para producir un resultado. Comprender los diferentes tipos de operadores es clave para escribir código eficaz y predecir su comportamiento.

1



Unarios

Actúan sobre **un solo** operando: `+`, `-`, `not`

Binarios

Actúan sobre **dos** operandos: aritméticos, relacionales, lógicos

Especiales

`in` (pertenencia) e `is` (identidad de objeto)

```
# Unarios
```

```
x = 10
```

```
y = -x # y = -10
```

```
# Binarios
```

```
suma = 5 + 3
```

```
es_igual = (5 == 3) # False
```

```
# Especiales
```

```
lista = [1, 2, 3]
```

```
print(2 in lista) # True
```

```
a = [1, 2]
```

```
b = a
```

```
c = [1, 2]
```

```
print(a is b) # True (mismo objeto)
```

```
print(a is c) # False (distinto objeto)
```

Jerarquía de Operadores

La **precedencia** determina el orden en que Python evalúa los operadores dentro de una expresión. Los operadores con mayor precedencia se evalúan primero. Conocer esta tabla evita errores sutiles y resultados inesperados en tu código.

Prioridad	Operadores	Descripción
1 (máxima)	()	Paréntesis (agrupación)
2	**	Exponente
3	+X, -X, ~X	Unarios
4	*, /, //, %	Multiplicación, división, módulo
5	+, -	Suma y resta
6	==, !=, <, >, <=, >=	Comparaciones
7	not	NOT lógico
8	and	AND lógico
9 (mínima)	or	OR lógico

Precedencia en Acción

Veamos cómo la jerarquía de operadores cambia radicalmente el resultado de una expresión. Los paréntesis son tu mejor herramienta para controlar el orden de evaluación y hacer tu código más legible.

Sin paréntesis

```
resultado = 5 + 3 * 2
# Paso 1: 3 * 2 = 6
# Paso 2: 5 + 6 = 11
print(resultado) # 11
```

La multiplicación tiene mayor precedencia que la suma, por lo que se evalúa primero.

Con paréntesis

```
resultado = (5 + 3) * 2
# Paso 1: 5 + 3 = 8
# Paso 2: 8 * 2 = 16
print(resultado) # 16
```

Los paréntesis fuerzan que la suma se evalúe antes que la multiplicación.



Asociatividad especial: El operador de exponente `**` se evalúa de **derecha a izquierda**. Así, `2 ** 3 ** 2` equivale a `2 ** (3 ** 2) = 2`
`** 9 = 512`, no a `(2 ** 3) ** 2 = 64`.

Operadores Aritméticos

Los operadores aritméticos son los más intuitivos: realizan las operaciones matemáticas que ya conoces. Python ofrece **siete operadores aritméticos**, incluyendo dos que no existen en una calculadora convencional: la división entera y el módulo.

Operador	Significado	Ejemplo	Resultado
+	Suma	5 + 3	8
-	Resta	5 - 3	2
*	Multiplicación	5 * 3	15
/	División (float)	5 / 2	2.5
//	División entera	5 // 2	2
%	Módulo (resto)	5 % 2	1
**	Potencia	5 ** 2	25

```
a = 10
b = 3
print(a + b) # 13
print(a / b) # 3.333...
print(a // b) # 3
print(a % b) # 1
print(a ** b) # 1000
```

Operadores Relacionales

Los operadores relacionales comparan dos valores y devuelven un resultado booleano (True o False). Son la base para tomar decisiones en el código: cada vez que usas un `if` o un `while`, estás usando una expresión relacional internamente.



`> y <`

Mayor que / Menor que. Comparan magnitud numérica o posición lexicográfica en cadenas.



`== y !=`

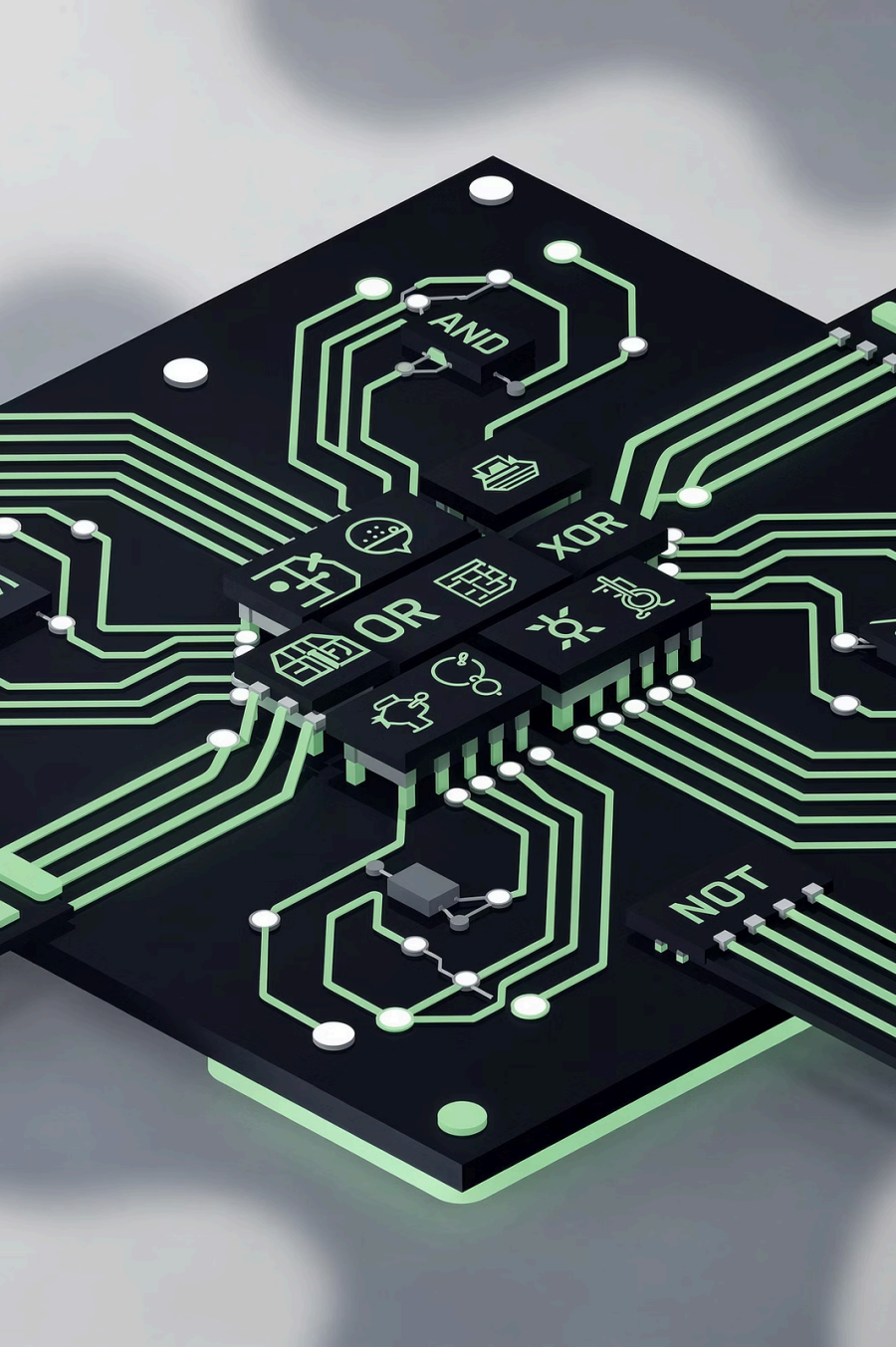
Igual a / Diferente de. Comparan si dos valores son equivalentes o no.



`>= y <=`

Mayor o igual / Menor o igual. Incluyen el caso de igualdad en la comparación.

```
edad = 18
print(edad >= 18) # True → puede votar
print(edad == 21) # False → no tiene 21 años
print(edad != 15) # True → es diferente de 15
```



① TEMA 2.9 · 30 MIN

Operadores Lógicos o Booleanos

Los operadores lógicos combinan expresiones booleanas para formar condiciones más complejas. Son indispensables cuando necesitas verificar **múltiples condiciones** simultáneamente. Python ofrece tres operadores principales: `and`, `or` y `not`.

`and`

Devuelve True solo si **ambas** condiciones son verdaderas. Ideal para verificar requisitos simultáneos.

`or`

Devuelve True si **al menos una** condición es verdadera. Útil para alternativas válidas.

`not`

Niega el valor booleano. Convierte True en False y viceversa.

Tablas de Verdad

Las tablas de verdad muestran todos los resultados posibles de una operación lógica. Dominar estas tablas te permite predecir el comportamiento de cualquier expresión booleana compuesta.

AND (and)

A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

OR (or)

A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

NOT (not)

A	not A
True	False
False	True

XOR (simulado con !=)

A	B	A != B
False	False	False
False	True	True
True	False	True
True	True	False

Operadores Lógicos en Práctica

Veamos cómo se combinan los operadores lógicos con los relacionales para construir condiciones reales. Recuerda que la precedencia importa: `not` se evalúa primero, luego `and`, y finalmente `or`.

```
x = 5
y = 10
z = 3

print(x < y and y > z)    # True and True = True
print(x > y or y < z)     # False or False = False
print(not (x == y))       # not False = True
print((x < y) != (y < z)) # True != False = True (XOR)
```



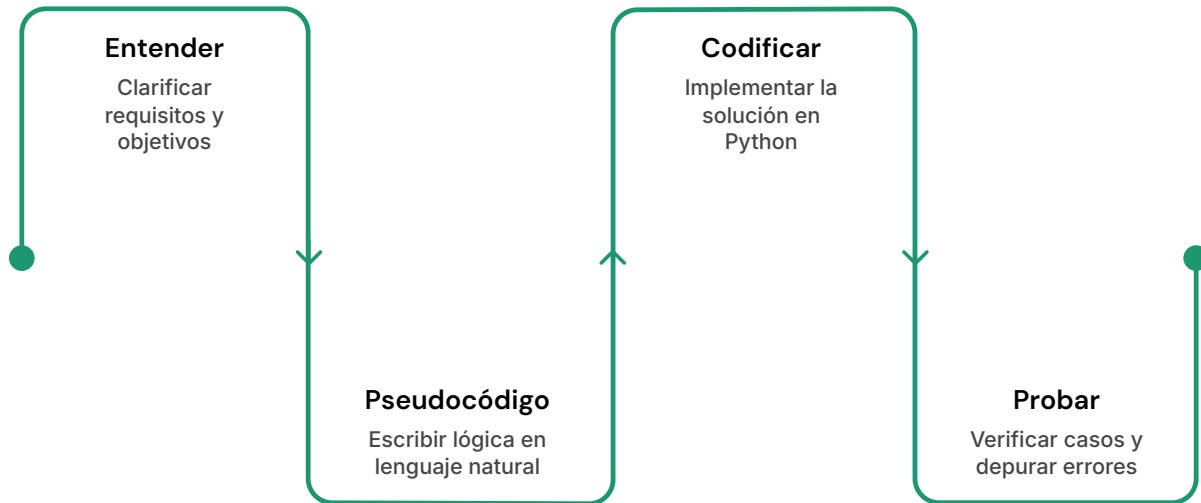
Desafío mental: Evalúa esta expresión paso a paso antes de ejecutarla en Python:

```
(5 > 3) and (10 < 20) or not (7 == 7)
```

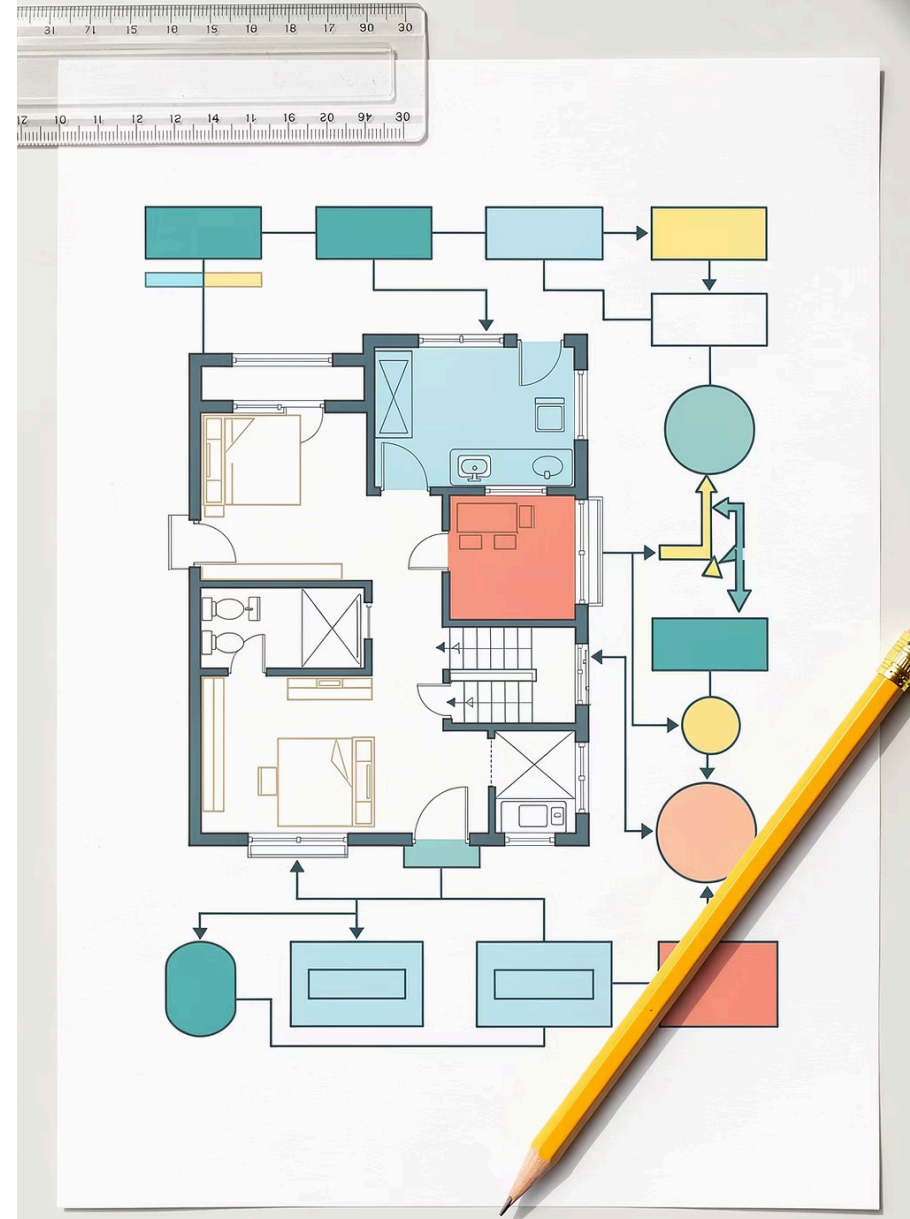
¿El resultado es `True` o `False`? Aplica la precedencia: primero `not`, luego `and`, y por último `or`.

Pseudocódigo y Algoritmos

Para cerrar el módulo, es fundamental mostrar cómo todos estos elementos se combinan para construir **algoritmos**: secuencias ordenadas de pasos que resuelven un problema. El **pseudocódigo** es una herramienta intermedia entre el lenguaje natural y el código real que te permite diseñar soluciones antes de programar.



Siguiendo este proceso, pasas de la comprensión del problema a una solución verificada, reduciendo errores y mejorando la calidad de tu código desde el primer intento.



Ejemplo: Área de un Rectángulo

Veamos un ejemplo completo donde aplicamos pseudocódigo y luego lo traducimos a Python. El problema es sencillo: calcular el área de un rectángulo dados su base y su altura.

Pseudocódigo

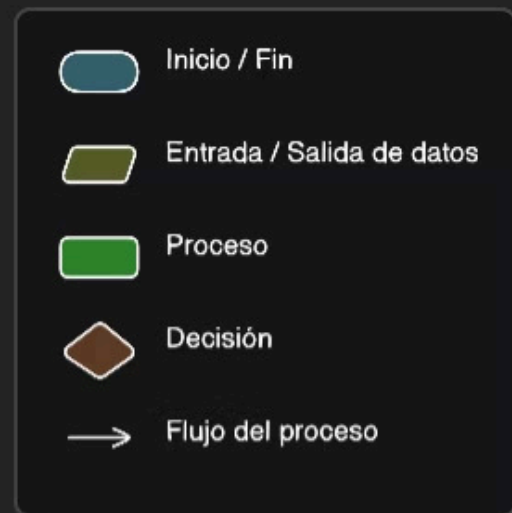
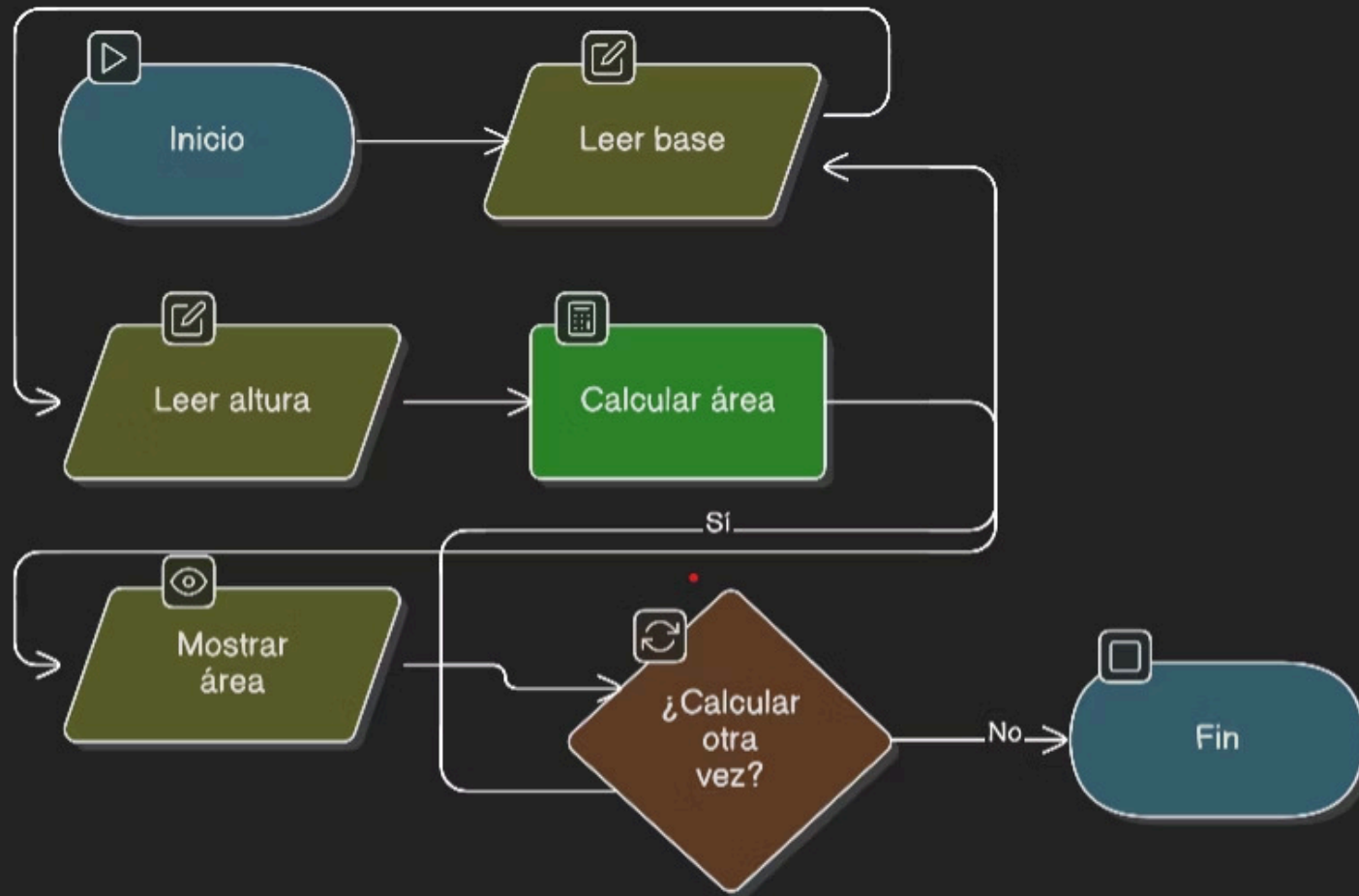
```
INICIO
  LEER base
  LEER altura
  area ← base × altura
  ESCRIBIR "El área es:", area
FIN
```

El pseudocódigo describe la lógica de forma clara y sin preocuparse por la sintaxis exacta de ningún lenguaje.

Implementación en Python

```
base = float(input("Base: "))
altura = float(input("Altura: "))
area = base * altura
print("El área es:", area)
```

Observa cómo la traducción es casi directa: cada línea del pseudocódigo tiene su equivalente en Python.



Actividades Prácticas

Pon a prueba todo lo aprendido con estos ejercicios progresivos. Cada uno refuerza un conjunto diferente de conceptos del módulo, desde los operadores básicos hasta el diseño de algoritmos completos.

1 Calculadora de operaciones (20 min)

Escribe un programa que solicite dos números al usuario y muestre su **suma, resta, multiplicación, división, división entera y módulo**. Usa `input()` y `float()` para la entrada.

3 Evaluación de expresiones (20 min)

Evalúa expresiones complejas respetando la **precedencia de operadores**: primero resuélvelas mentalmente en papel y luego comprueba el resultado en Python.

2 Laboratorio booleano (20 min)

Crea variables booleanas a partir de **expresiones relacionales** y combínalas con `and`, `or` y `not`. Muestra los resultados con `print()` y verifica las tablas de verdad.

4 Proyecto corto (tiempo restante)

Diseña en **pseudocódigo** y luego implementa en Python un algoritmo que resuelva: calcular el promedio de 3 calificaciones o determinar si un número es par o impar.